Week 5 Practice Tasks

For each task, implement the requested method or repair the supplied code. Predict the example result before running it, then check the example and at least one boundary case. Answer the “Explain” question in 1–3 sentences. Keep console input and output in Main unless a task explicitly studies side effects.

Use a C# console project with System, System.Collections.Generic and System.Linq. The model on page 5 uses records, supported by .NET 8. Method signatures in the exercises omit public static for readability; add those modifiers when placing methods in Program.

Pure functions and side effects

A pure calculation depends only on its inputs and produces no observable side effects. It does not print, read the console, modify an input collection or update shared state. Local variables and loops are allowed: changing a local total does not itself make a function impure.

```
static int AddBonus(int score, int bonus) => score + bonus;

// Keep output at the edge of the program.
int result = AddBonus(12, 3);
Console.WriteLine(result); // 15
```

A **method that prints the same answer every time still has a side effect**. A method that reads a changing global variable also depends on more than its explicit arguments. Separate calculation from those effects so you can test it with simple inputs and expected results.

Functions passed as arguments

```
static int Apply(int value, Func<int, int> rule)
{
    return rule(value);
}

int a = Apply(4, x => x + 3);  // 7
int b = Apply(4, x => x * 2);  // 8
```

Func<int, int> means “accept an int and return an int”. The last type argument is the return type. The lambda x => x * 2 is a function: x is its parameter and x * 2 is its result. Passing rule does not call it; rule(value) calls it. Action<T> is used for a function with no return value, such as displaying a value.

Decisions and sequence processing

Switch expressions and validation

```
static int LevelBonus(int level) => level switch
{
    1 => 5,
    2 => 10,
    3 => 20,
    _ => 0
};
```

A switch expression selects a value. The discard pattern _ handles remaining cases. Decide from the contract whether an unknown input should produce a fallback or be rejected; do not silently invent a valid value for invalid data.

```
bool ok = int.TryParse(text, out int value)
    && value >= 1 && value <= 6;
```

TryParse checks whether text represents an integer; it does not check a business range. The two comparisons enforce the inclusive limits. With &&, all requirements must hold. With ||, one true requirement is enough. Short-circuit evaluation means the right side of && is skipped when the left side is false.

Return and continue

return ends the whole method and optionally returns a value. continue skips the rest of the current loop iteration and moves to the next one. If one invalid line should be ignored while later lines are processed, leaving the whole method is usually the wrong action.

IEnumerable

IEnumerable<T> describes something that can be enumerated to obtain values of type T. It may be an array, a list, a LINQ query or an iterator. It does not promise indexing, a stored collection or a fixed amount of data. Do not assume that every source supports a second traversal.

```
static IEnumerable<int> UpTo(int limit)
{
    for (int n = 1; n <= limit; n++)
        yield return n;
}
```

Calling UpTo(3) creates a sequence object. The iterator body starts when the consumer requests the first element. yield return gives the consumer one value and suspends the iterator. A later request resumes after that yield. yield break ends the iterator; reaching the end also ends it.

If no yield return is reached, enumeration produces no elements. For this iterator, UpTo(0) is an empty sequence, not null. A consumer may stop early, so code after the last yielded value it consumes may never execute.

Neighbouring values

For adjacent pairs, keep a previous value and a flag indicating whether one exists. The first input establishes the state. Each later input forms a pair with the previous one, then becomes the new previous value. For [4, 9, 2], the pairs are (4, 9) and (9, 2). A sequence of n values has max(n − 1, 0) adjacent pairs.

LINQ

LINQ provides operations for querying sequences. Read a method chain from left to right: at each step, ask what one element looks like and which elements can continue. Most sequence operations describe work that will happen when the result is enumerated.

METHOD	PURPOSE	IMPORTANT DETAIL
WHERE(PREDICATE)	Keep matching elements	Element type stays the same.
SELECT(SELECTOR)	Transform each element	The output element type may change.
ORDERBY / ORDERBYDESCENDING	Set the primary ordering	Ascending / descending.
THENBY / THENBYDESCENDING	Add a tie breaker	Preserves the primary ordering.
TAKE(N) / SKIP(N)	Keep / skip an initial part	Operate on the order at that point.
ANY / ALL	Test existence / every element	Empty: Any() is false; All(test) is true.
COUNT / SUM	Count elements / add numbers	Produce a result immediately.
FIRSTORDEFAULT	First match or default value	For int, default is 0; this may be ambiguous.
TOARRAY / TOLIST	Materialize the current results	Enumerate now and store the results.
AGGREGATE(SEED, (ACC, X) => FUNC)	Combines elements into one accumulated result	Updates the accumulator for each element. For empty input, returns the seed if provided; otherwise, throws an exception.

```
int[] values = { 5, 2, 8, 1, 4 };
int[] result = values
   .Where(n => n % 2 == 0)  // 2, 8, 4
   .Select(n => n * 10)    // 20, 80, 40
   .OrderByDescending(n => n) // 80, 40, 20
   .Take(2)              // 80, 40
   .ToArray();
```

Where decides whether an element stays; Select decides what it becomes. Take(2) returns at most two elements. If fewer are available, it returns those elements. Take(0) produces an empty sequence. Placing Take before sorting may discard the record that should rank first.

Keep the original object and a calculated value

```
var scored = tickets.Select(t => new
{
   Item = t,
   Score = rule(t)
});
var ordered = scored.OrderByDescending(x => x.Score)
   .ThenBy(x => x.Item.Id);
```

Before Select, each element is a ticket. After Select, each element is a new anonymous object with Item and Score properties. Item holds the ticket, so **its ID** is **x.Item.Id**. Score holds the calculated integer, so it is x.Score. Renaming the lambda parameter does not change its type.

Store the score once when it is needed for both ordering and output. ThenBy resolves equal primary keys. A second OrderBy creates a new primary ordering instead of adding a secondary key.

When the work actually happens

An iterator and a LINQ query are often deferred: creating the query is not the same as computing its results. foreach, ToArray and terminal operations such as Count or Sum request values. Short-circuiting consumers such as Any or Take may finish without reading the entire source.

```
static IEnumerable<int> Source()
{
   for (int n = 1; n <= 5; n++)
   {
      Console.WriteLine($"read {n}");
      yield return n;
   }
}

var query = Source().Where(n => n % 2 == 0).Take(1);
Console.WriteLine("created");
foreach (int n in query) Console.WriteLine($"got {n}");
```

Output: created, read 1, read 2, got 2. Source is not read when query is assigned. During the loop, Where rejects 1 and accepts 2. Take has obtained one matching element and stops. Values 3–5 are not needed.

Deferred does not always mean one input per output

Where can inspect several inputs before finding one match. An adjacent-pair iterator needs two inputs for its first result. Ordering a general finite source needs to inspect the whole source before it can know which element comes first. OrderByDescending(...).Take(1) therefore cannot simply stop at the first input.

Repeating a query and storing a snapshot

```
var numbers = new List<int> { 1, 2, 3 };
var query = numbers.Where(n => n >= 2);
int[] saved = query.ToArray();  // stores 2, 3
numbers.Add(4);
Console.WriteLine(string.Join(", ", query)); // 2, 3, 4
Console.WriteLine(string.Join(", ", saved)); // 2, 3
```

Re-enumerating this query reads the list again. saved already contains the values obtained earlier. ToArray stores the sequence results; it does not deeply clone objects contained in those results. A query can also capture a variable, such as a threshold, and read its value when enumeration happens.

A repeatable way to solve a task

First state the input, output and boundary cases. Next decide what must be remembered or calculated. Write the smallest correct method, then check a normal case, an empty case where applicable, and a boundary or tie case. For a LINQ pipeline, write the element shape after each Select. For an iterator, trace its local state after every input.

Do not put side effects inside filtering or scoring lambdas in ordinary calculations. The Source example prints deliberately so you can observe execution. Such tracing is a teaching tool, not a pure transformation.

Shared model and example data

!!! Use this model in the ticket exercises. Tasks that use only integers or strings do not need it. Copy the data into your project once. The row order shown here is the input order, not the expected report order.

```
public enum TicketKind { Question, Request, Incident }

public record Ticket(
    int Id, string Title, int Effort,
    bool IsOpen, TicketKind Kind);

static Ticket[] SampleTickets() => new[]
{
    new Ticket(8, "WiFi", 3, true, TicketKind.Incident),
    new Ticket(3, "Login", 3, true, TicketKind.Incident),
    new Ticket(5, "Printer", 4, true, TicketKind.Request),
    new Ticket(1, "Server", 7, true, TicketKind.Incident),
    new Ticket(7, "Email", 2, false, TicketKind.Incident),
    new Ticket(6, "Account", 2, true, TicketKind.Question)
};
```

Common assumptions

Unless an exercise asks you to validate input, assume non-null sequences, non-null delegates and non-null model objects. Ticket IDs are unique and positive, titles are non-null, Effort is from 1 to 8, and Kind is a declared enum value. Rules and predicates are pure unless a tracing exercise deliberately says otherwise.

Integer sequence values are between −1000 and 1000, and ordinary exercise sequences have at most 100 elements. Integer overflow is outside the scope of these tasks. Brackets such as [2, 4] describe sequence contents; they are not a required syntax for constructing arrays.

Exercise route

Tasks	Focus	What to practise
01–05	Functions and lambdas	Return values, pure calculations, delegate calls.
06–10	Debugging and boundaries	Validation, loop control, counters, state.
11–16	Lazy iterators	yield return, local state, early termination.
17–24	LINQ operations	Filtering, projection, ordering, materialization.
25–30	Combining techniques	Diagnose pipelines and build complete transformations.

For debugging exercises, keep the broken version long enough to show a failing check. Then repair it and show that the same check passes. Matching one example is not sufficient: read the complete contract and explain why your approach handles it.

Exercises 01 to 03

01 Separate calculation from output

Implement a pure function that adds a fixed fee of 5 to a non-negative amount. Then call it from Main and print its returned value. The calculation method must not read or print anything.

int AddFee(int amount)

Example. AddFee(12) → 17; AddFee(0) → 5.

Check. Call the method twice with 12 and compare the results.

Explain. Why does moving Console.WriteLine to Main make the calculation easier to test?

02 Choose a value with a switch expression

Return a bonus for a TicketKind: Question gives 5, Request gives 10, and Incident gives 20. Use a switch expression. For any undeclared enum value, throw ArgumentOutOfRangeException.

int KindBonus(TicketKind kind)

Example. Question → 5; Request → 10; Incident → 20.

Check. Also call the function with (TicketKind)99 and verify rejection.

Explain. Why can an enum variable still require validation even though its type is an enum?

03 Use the function supplied by the caller

Implement Apply so that it invokes rule once with value and returns the result. In Main, use one lambda that adds 7 and another that multiplies by 3. Do not hard-code either calculation inside Apply.

int Apply(int value, Func<int, int> rule)

Example. Apply(4, x => x + 7) → 11; Apply(4, x => x * 3) → 12.

Check. Apply(4, x => 100) must return 100.

Explain. What is the difference between passing rule and writing rule(value)?

Exercises 04 to 06

04 Apply two functions in order

Apply first to the input and then apply second to that result. Return the final integer. Keep the method pure and invoke each supplied function once.

```
int ApplyInOrder(int value, Func<int, int> first,
   Func<int, int> second)
```

Example. Input 4, first x => x + 2, second x => x * 3 → 18. Reversing the rules gives 14.

Check. Use the identity lambda x => x as one of the two rules.

Explain. Why does changing the order of the functions change the result?

05 Count using a predicate

Use one foreach loop to count values accepted by the supplied predicate. Do not use LINQ. In Main, call the same method once for positive values and once for even values.

```
int CountMatching(IEnumerable<int> values, Func<int, bool> test)
```

Example. For [-2, 0, 3, 4], x => x > 0 gives 2; x => x % 2 == 0 gives 3.

Check. Empty input → 0; a predicate that always returns false → 0.

Explain. What does the bool returned by the predicate tell the caller?

06 Repair an inclusive range check

The method must accept only integers from 1 to 6 inclusive. Null, blank, non-integer and out-of-range input must return false. Repair the condition, retaining TryParse and the out parameter. The value output is meaningful only when the method returns true.

```
bool TryRead(string text, out int value)
{
   return int.TryParse(text, out value)
      && value > 1 && value < 6;
}
```

Example. "1" and "6" → true; "0", "7", "abc" and null → false.

Check. Include both boundary values and one valid middle value.

Explain. Which checks belong to TryParse, and which checks belong to the task's range rule?

Exercises 07 to 09

07 Skip bad input and keep adding

Sum every string that can be parsed as an integer. Ignore invalid lines and continue. Return 0 for an empty input. There are two logic errors in the method below. Repair both, then create one small diagnostic check for each error.

```
int SumValid(IEnumerable<string> lines)
{
   int total = 0;
   foreach (string line in lines)
   {
      if (!int.TryParse(line, out int n)) return total;
      total = n;
   }
   return total;
}
```

Example. ["3", "bad", "5", "-2"] → 6.

Check. Use one test with invalid input first and another with two valid numbers.

Explain. Why does return at the first invalid line make later valid values disappear?

08 Repair the smallest positive value

Return the smallest strictly positive integer, or -1 if none exists. Diagnose and repair the state initialization and update logic below. Use one pass and no LINQ.

```
int SmallestPositive(IEnumerable<int> values)
{
   int best = 0;
   foreach (int n in values)
      if (n > 0 && n < best) best = n;
   return best;
}
```

Example. [8, -2, 3, 5] → 3; [0, -4] → -1; [] → -1.

Check. Check a single positive value and descending positive values.

Explain. Why is 0 a poor initial minimum when all candidate values must be positive?

09 Require both conditions and count every match

Count open tickets whose Effort is at most maxEffort. maxEffort is from 1 to 8. Repair the two faulty lines below inside an otherwise correct foreach over tickets. Write two tests that isolate the two errors.

```
if (t.IsOpen || t.Effort <= maxEffort)
   count = 1;
```

Example. SampleTickets() with maxEffort = 4 → 4.

Check. Test a closed low-effort ticket alone, and two eligible tickets together.

Explain. Why does using || accept records that satisfy only half of the requirement?

Exercises 10 to 12

10 Remove input mutation

Refactor this method so it returns a new array containing the original values followed by extra. Accept IEnumerable<int> instead of List<int>. Keep the input unchanged and remove console output. A local List<int> is allowed for this exercise.

```
int[] AppendValue(List<int> values, int extra)
{
   values.Add(extra);
   Console.WriteLine(values.Count);
   return values.ToArray();
}
```

Example. Input list [2, 4], extra 7 → [2, 4, 7]; original list remains [2, 4].

Check. Call twice with the same list; it must not grow.

Explain. How is changing a local result collection different from changing the caller's list?

11 Generate a lazy inclusive range

Use yield return to produce every integer from start to end inclusive. If start is greater than end, produce an empty sequence. Bounds are within −1000 to 1000. Do not build a collection or use LINQ.

```
IEnumerable<int> InclusiveRange(int start, int end)
```

Example. (2, 5) → [2, 3, 4, 5]; (3, 3) → [3]; (5, 2) → [].

Check. Try a range that crosses zero.

Explain. Does the loop run when the method is called or when its result is enumerated?

12 Filter lazily using a function

Implement a lazy filter using foreach and yield return. Call test once for each inspected element and preserve input order. Do not use Where, a temporary collection or repeated traversal.

```
IEnumerable<int> Keep(IEnumerable<int> values,
   Func<int, bool> test)
```

Example. [-2, 5, 0, 8] with x => x > 0 → [5, 8].

Check. Empty input and an always-false predicate must produce empty sequences.

Explain. If the consumer requests only the first result, must Keep inspect every input value?

Exercises 13 to 15

13 Calculate adjacent signed differences

For each overlapping adjacent pair, yield current minus previous. Use one traversal, fixed local storage and no LINQ, indexing or temporary collection. Empty and single-element inputs produce an empty sequence.

IEnumerable<int> AdjacentChanges(IEnumerable<int> values)

Example. [12, 7, 10, 10] → [-5, 3, 0].

Check. [-3, 2, -5] → [5, -7]; [7] → [].

Explain. Which input values are needed to produce the first result, and why is the result not null for [7]?

14 Find each adjacent maximum

Yield the greater value from each overlapping neighbouring pair. This is a pairwise maximum, not the maximum of the entire prefix. Use one pass, yield return and fixed local storage; do not use LINQ.

IEnumerable<int> AdjacentMaximums(IEnumerable<int> values)

Example. [2, 9, 4, 4] → [9, 9, 4].

Check. [9, 2, 1] → [9, 2]; empty and single-element inputs → [].

Explain. Why would remembering the greatest value seen so far give the wrong final result?

15 Produce running totals

Yield a cumulative sum after reading each input value. Use one pass and a local accumulator. Do not modify the source or use a shared field, LINQ or a temporary collection.

IEnumerable<int> RunningTotals(IEnumerable<int> values)

Example. [3, -1, 4] → [3, 2, 6]; [] → [].

Check. Enumerate the result twice when the source is an array; each pass should begin from zero.

Explain. Why should the accumulator belong to this iterator execution rather than a static field?

Exercises 16 to 18

16 Stop at a sentinel

Yield values until the first negative value is encountered. Do not yield that negative value or inspect later input values. Zero is valid. Use yield return and yield break, with no LINQ or temporary collection.

IEnumerable<int> BeforeNegative(IEnumerable<int> values)

Example. [2, 0, 5, -1, 8] → [2, 0, 5]; [-1, 8] → [].

Check. An input with no negative numbers returns all its values.

Explain. How does stopping at a negative value differ from filtering out every negative value?

17 Filter numbers with LINQ

Return an array of even values that are at least min. Use Where and ToArray, and preserve the original order. min is between −1000 and 1000. Do not sort the result.

int[] EvenAtLeast(IEnumerable<int> values, int min)

Example. [5, 2, 8, 4, 1], min = 4 → [8, 4].

Check. A value equal to min must be included when it is even; empty input → [].

Explain. Why does this task need Where rather than Select?

18 Transform values into labels

Use Select to turn each integer into a string in the exact format Value=number. Keep all elements in their original order and return a string array.

string[] LabelValues(IEnumerable<int> values)

Example. [-2, 0, 7] → ["Value=-2", "Value=0", "Value=7"].

Check. Empty input → an empty string array.

Explain. What is the element type before Select and after Select?

Exercises 19 to 21

19 Select IDs from matching tickets

Return the IDs of open Incident tickets with Effort at most maxEffort. Use LINQ and preserve their input order. maxEffort is from 1 to 8.

int[] IncidentIds(IEnumerable<Ticket> items, int maxEffort)

Example. SampleTickets(), maxEffort = 4 → [8, 3].

Check. With maxEffort = 2, the result is empty: the Email ticket is closed.

Explain. Why is it easier to filter by ticket properties before selecting only the ID?

20 Sort with a tie breaker

Sort all tickets by Effort ascending, then by Id descending when efforts are equal. Return their IDs as an array. Do not filter out closed tickets.

int[] OrderedIds(IEnumerable<Ticket> items)

Example. SampleTickets() → [7, 6, 8, 3, 5, 1].

Check. Use two equal-effort tickets whose IDs arrive in ascending order.

Explain. What requirement would be broken by replacing ThenByDescending with a second OrderByDescending?

21 Take a page after sorting

Sort all tickets by Id ascending, skip skip records and return up to count remaining IDs. skip and count are non-negative. Use OrderBy, Skip, Take and Select.

int[] PageIds(IEnumerable<Ticket> items, int skip, int count)

Example. SampleTickets(), skip = 2, count = 3 → [5, 6, 7].

Check. count = 0 → []; skip beyond the source length → []; excessive count returns the remaining IDs.

Explain. Why must pagination operate on a defined order?

Exercises 22 to 24

22 Compute a small summary

Using LINQ, implement four methods over tickets: HasOpen returns whether any ticket is open; AllWithin tests whether every Effort is <= maxEffort; OpenCount counts open tickets; OpenEffortTotal sums their Effort. Do not change the source. Assume a repeatable finite input.

Example. SampleTickets(), maxEffort = 4 → HasOpen true, AllWithin false, OpenCount 5, OpenEffortTotal 19.

Check. On empty input, expect false, true, 0 and 0 respectively.

Explain. Why is AllWithin true for an empty input, while HasOpen is false?

23 Preserve the ticket and its score

Keep open tickets. Project each into an anonymous object with Item and Score, where Score is calculated by rule. Sort by Score descending and Id ascending; return every line as Id: Title [score]. Reuse Score for formatting and invoke rule once per included ticket.

```
string[] ScoredLines(IEnumerable<Ticket> items,
   Func<Ticket, int> rule)
```

Example. SampleTickets(), rule t => t.Effort: first two lines are "1: Server [7]" and "5: Printer [4]"; there are 5 lines.

Check. With t => 100, IDs must appear as 1, 3, 5, 6, 8.

Explain. Why is the ID read through x.Item.Id after the anonymous-object projection?

24 Observe deferred execution

Reuse Source from page 4. Predict the complete output, then run three queries separately: A is Source().Where(n => n % 2 == 0).Take(2); B is Source().OrderByDescending(n => n).Take(1); C materializes A with ToArray and then loops over that array twice. Print got n for each consumed value.

Example. A produces values [2, 4]; B produces [5]; the saved array in C produces [2, 4] on both loops.

Check. A reads source values 1–4; B reads 1–5; C reads 1–4 once during ToArray and does not read Source during either array loop.

Explain. Why can A stop early while B needs the complete input?

Exercises 25 to 27

25 Repair a ranking pipeline

The required result is the IDs of the top count open tickets by Effort descending, breaking ties by Id ascending. count is non-negative. Repair the two ordering-related errors in this pipeline. Keep the return type int[].

```
return items.Where(t => t.IsOpen).Take(count)
   .OrderByDescending(t => t.Effort)
   .OrderBy(t => t.Id)
   .Select(t => t.Id).ToArray();
```

Example. SampleTickets(), count = 2 → [1, 5].

Check. Use count = 1 to expose early limiting; use count = 10 to expose the wrong primary order.

Explain. Explain separately why the position of Take is wrong and why the second OrderBy is wrong.

26 Parse valid values lazily

Read strings lazily. Yield only parsed integers from 1 to 6 inclusive. Skip invalid, blank, null and out-of-range lines, then continue with later lines. Use TryParse, a single foreach and yield return. Do not use LINQ or a temporary collection.

```
IEnumerable<int> ValidValues(IEnumerable<string> lines)
```

Example. ["2", "bad", "6", "0", null, " 3 "] → [2, 6, 3].

Check. ["bad", "4"] → [4]; ["1", "6"] → [1, 6]; all invalid → [].

Explain. Why does validation need both successful parsing and a range check, and why should invalid input not end the iterator?

27 Combine parsing and adjacent changes

Reuse ValidValues from Task 26 and AdjacentChanges from Task 13. Parse valid values first, compute changes between neighbouring valid values, keep strictly positive changes, sort descending and return up to count changes as an int array. count is non-negative. Use LINQ after the iterators.

```
int[] LargestIncreases(IEnumerable<string> lines, int count)
```

Example. ["1", "bad", "4", "2", "6"], count = 2 → [4, 3]. Valid values are [1, 4, 2, 6].

Check. One valid value → []; no positive changes → []; count = 0 → [].

Explain. Why do 1 and 4 become neighbours even though an invalid line appeared between them?

Exercises 28 to 30

28 Change strategy without changing the report

Implement CheapestIds: keep open tickets with Effort <= maxEffort, apply the supplied cost rule at most once per eligible ticket, order cost ascending then Id ascending, and return up to count IDs. Implement two pure cost lambdas in Main: Effort; and 10 − Effort. maxEffort is 1–8 and count is non-negative.

```
int[] CheapestIds(IEnumerable<Ticket> items,
    Func<Ticket, int> cost, int maxEffort, int count)
```

Example. SampleTickets(), maxEffort = 4, count = 2: t => t.Effort gives [6, 3]; t => 10 - t.Effort gives [5, 3].

Check. With t => 100, the same inputs give [3, 5].

Explain. Why should CheapestIds not contain a switch that chooses between the two cost rules?

29 Predict a captured variable and a snapshot

Predict both printed lines before running this code. Then change it so the first query keeps using the original threshold even after min changes. Store a separate original threshold before creating that query; keep the later source addition.

```
var values = new List<int> { 1, 3, 5 };
int min = 3;
var query = values.Where(x => x >= min);
var saved = query.ToArray();
min = 5;
values.Add(7);
Console.WriteLine(string.Join(", ", query));
Console.WriteLine(string.Join(", ", saved));
```

Example. Original output: "5, 7" then "3, 5". After your threshold change: "3, 5, 7" then "3, 5".

Check. Explain why preserving the threshold still allows the new list element 7 to appear.

Explain. Which values are stored by ToArray, and which variable does the deferred predicate read later?

30 Build and verify a complete report

Return up to count strings formatted Id: Title [score]. Keep open tickets with Effort <= maxEffort, use the supplied rule, sort by score descending and Id ascending, then limit and format. Cache each score for reuse; never score excluded tickets. Use LINQ, no input mutation or I/O. maxEffort is 1–8; count is non-negative.

```
string[] BuildReport(IEnumerable<Ticket> items,
    Func<Ticket, int> rule, int maxEffort, int count)
```

Example. SampleTickets(), rule t => t.Effort * 2 + (t.Kind == TicketKind.Incident ? 5 : 0), maxEffort = 4, count = 3 → ["3: Login [11]", "8: WiFi [11]", "5: Printer [8]"].

Check. Write checks for the example, count 0, no matches, an excessive count and an all-equal rule. Explain which requirement each check covers.

Explain. Describe the element type and purpose at each stage: filter, score projection, ordering, limiting, string projection and materialization.